

Comprehensive Report: Market Basket Analysis & Baseline Recommendation Strategy

1. Executive Summary

This report presents the methodological framework and execution pipeline for the H&M associative and cold-start recommendation engines. Operating on a highly sparse dataset of ~31.7M transactional records, the primary challenge was circumventing severe Out-Of-Memory (OOM) constraints while extracting valid cross-selling associations.

Through the strategic application of native mathematical pruning, a Dual-Window temporal heuristic, and the computationally efficient FP-Growth algorithm, the pipeline successfully extracted high-confidence multi-item rules. Furthermore, the system transitions from purely predictive analytics to **prescriptive analytics** by integrating RFM segmentation, prioritizing higher-price/high-AOV bundles for premium VIP segments to actively maximize the Average Order Value (AOV).

2. Available Datasets & Pipeline Execution

The pipeline inherits cleansed and segmented artifacts from the upstream Data Engineering (Person A) and Descriptive Analytics (Person B) phases. To process the massive data volume without localized RAM failure, the entire ingestion framework was built upon the **Polars Lazy API** (`pl.scan_parquet`), deferring computation until mathematically necessary.

Input Artifacts: * `train_processed.parquet`: The chronologically partitioned interaction matrix. Used to form authentic shopping baskets and calculate item frequencies. * `customers_segmented.parquet`: RFM profiles containing `age` (for cold-start demographics) and `segment_label` (for prescriptive VIP ranking). * `articles_processed.parquet`: Master catalog metadata, utilized to extract `product_group_name` and calculate the `median_price` proxy.

3. Dimensionality Reduction & Algorithmic Optimization

Standard frequent pattern mining on fast-fashion datasets introduces catastrophic memory spikes due to the “long tail” of sparse items. The following optimizations were rigorously applied prior to algorithmic execution:

3.1. The Dual-Window Heuristic (Dead-Stock Prevention)

- **Action:** Established an *Active Window* (last 30 days) to identify “alive” inventory, and a *Historical Window* (last 90 days) restricted exclusively to baskets containing these active items.
- **Purpose:** Align the data architecture with the ~90-day fast-fashion shelf-life discovered during EDA.
- **Impact:** Solves the “Dead-Stock Trap”. The algorithm successfully captures deep behavioral patterns (90 days) while utilizing the 30-day window as a robust proxy for the active catalog, reducing the risk of recommending obsolete or out-of-stock products.

3.2. Native Polars Mathematical Pruning

- **Action:** Pre-calculated the absolute `min_support` threshold (0.1%) and filtered non-frequent items entirely within the Polars Lazy evaluation graph (C++ backend) prior to matrix instantiation.
- **Purpose:** Bypass the critical flaw in the `mlxtend` framework, which internally densifies sparse matrices and attempts to allocate massive contiguous memory blocks (originally causing a >3GB RAM crash).
- **Impact:** Drastically shrank the matrix dimensionality. The system securely executed on localized hardware with zero memory spikes, avoiding the arbitrary and unscientific truncation of data (e.g., blindly capping at “Top 5000 items”).

3.3. Transaction ID (TID) Integrity

- **Action:** Constructed shopping baskets by grouping `[customer_id, t_dat]` and enforcing a strict `.unique()` array aggregation, subsequently dropping all single-item baskets.
- **Purpose:** Resolve a structural artifact where upstream deduplication collapsed identical SKUs into a `quantity` feature. A standard list aggregation would have generated false “multi-item” baskets (e.g., `[Item_A, Item_A]`).
- **Impact:** Ensures that the FP-Growth algorithm exclusively mines authentic *cross-item* correlations, preserving the statistical purity of the resulting Association Rules.

4. Frequent Pattern Mining & Rule Extraction

Following the generation of frequent itemsets, association rules were extracted and enriched to serve as the predictive core of the recommendation engine.

4.1. Stringent Statistical Thresholding

To guarantee that the generated recommendations represent true, high-probability cross-selling opportunities rather than random co-occurrences, two strict statistical boundaries were enforced: * **Confidence ≥ 0.20 :** Ensures a minimum 20% empirical probability that a customer will purchase the consequent given the antecedent. * **Lift > 1.0 :** Strictly filters for positive correlation, isolating items that have a synergistic purchasing relationship.

4.2. N-to-1 Rule Architecture & Serialization

The extraction logic was explicitly constrained to **N-to-1 mappings** (where the antecedent can contain multiple items, but the consequent is strictly a single item). This design enables the engine to evaluate complex multi-item shopping carts. Furthermore, native `frozenset` objects generated by `mlxtend` were programmatically unpacked into integer lists and scalars to ensure seamless compatibility with the Parquet serialization format.

4.3. Financial Proxy Derivation

Association rules inherently provide statistical correlation but lack financial context. To empower prescriptive business logic in subsequent steps, a `median_price` proxy was dynamically computed from the active 90-day transactional window for each `consequent_item`.

5. The Recommendation Engine Architecture

The final component is a dynamic, multi-layered Python function designed to handle cold-start scenarios, interpret complex shopping carts, and bridge predictive rules with prescriptive business strategy.

5.1. Demographically-Aware Cold-Start Initialization

For newly acquired users lacking transactional history (or entering with an empty cart), Collaborative Filtering fails. To resolve this, a **Demographic Prior** was constructed: * Users are classified into discrete chronological cohorts (`<25`, `25-35`, `36-50`, `>50`) utilizing RFM metadata (Person B). * A localized popularity matrix is generated by querying exclusively the **Active Window (last 30 days)**, ensuring cold-start recommendations are culturally relevant, highly-trending, and explicitly in-stock for that specific age group.

5.2. Advanced Cart-Based Matching

Moving beyond rigid 1-to-1 mappings, the engine utilizes subset evaluation (`(set(antecedents).issubset(cart_set))`) to analyze the user’s *entire* current shopping cart. This enables the engine to trigger N-to-1 associations (e.g., if a user adds both a blazer and trousers, the algorithm recognizes the combined antecedent and recommends a statistically correlated tie).

5.3. Prescriptive VIP Optimization (ECR heuristic)

Fulfilling the cross-check integration duty with Descriptive Analytics, the engine dynamically alters its ranking heuristic based on the user’s RFM Segment: * **Standard Users:** Recommendations are ranked by strict statistical `Lift`, optimizing purely for conversion likelihood. * **VIP Segments (“Mature Champions”, “Young Whales”):** The ranking mechanism shifts to prioritize the **Expected Cross-Sell Revenue (ECR)**.

$$ECR = Lift \times Confidence \times Consequent_Median_Price$$

By prioritizing the ECR metric, the algorithm prescriptively pushes higher-price/premium bundles to whales, maximizing the Average Order Value (AOV) and preventing the “ROI Trap” of suggesting low-cost, high-volume items to premium buyers.

5.4. Repurchase Tolerance & Fallback Padding

Empirical analysis of H&M’s ecosystem indicates that *repurchase* (e.g., rebuying basics/underwear) is a dominant behavioral signal. Consequently, the algorithm is explicitly designed to tolerate historical repurchases, filtering out only the items currently present in the active cart. A dynamic padding mechanism ensures the engine consistently outputs a robust vector of exactly $K = 12$ items by appending demographic-trending artifacts when rule-based consequents are exhausted.

6. Data Dictionary: Association Rules Output

The fully enriched, descending-sorted rule dataframe is exported as `Association_Rules.parquet`. This highly compressed artifact serves as the definitive data source for downstream Business Intelligence Dashboards (Person E).

Feature Name	Type	Description
<code>antecedents_list</code>	List[Int64]	Array of article IDs representing the required condition (Items in Cart).
<code>consequent_item</code>	Int64	The target article ID recommended for cross-selling.
<code>antecedent support</code>	Float64	The probability of the antecedent itemset occurring in the dataset.
<code>consequent support</code>	Float64	The probability of the consequent item occurring in the dataset.
<code>support</code>	Float64	The probability of both antecedent and consequent occurring together.
<code>confidence</code>	Float64	The conditional probability of buying the consequent given the antecedent.
<code>lift</code>	Float64	The strength of association. Values > 1.0 indicate positive correlation.

Feature Name	Type	Description
consequent_median_price	Float64	The normalized median price of the recommended item (AOV Proxy).
consequent_product_group	String	The macro-category of the recommended item (e.g., Garment Upper body).